DWC · 2DW3 · EXAMEN PROCEDIMENTAL · 2025-26

Guía práctica de JavaScript para el examen

Arrays · Objetos y clases · DOM y eventos · Validación de formularios · Web Storage

Plantillas listas para copiar y pegar · Errores que restan puntos · Chuleta de consulta rápida

00 Cómo usar esta guía

Esto no es teoría para leer del tirón. Es un **manual de plantillas**: buscas el ejercicio que te ha tocado, copias el bloque completo, cambias los nombres y lo pruebas. Todo el código de aquí está probado y funciona tal cual.

El examen siempre es lo mismo

Cambian los datos, los `id` y el texto de los mensajes. La estructura de cada ejercicio no cambia.

- 1 Recorrer / filtrar / buscar un array.
- 2 Clase con constructor y un método.
- 3 Botón que crea `<li>` y efecto hover.
- 4 Submit + regex + mensaje de error/éxito.
- 5 Login que guarda en `localStorage`.

Los 3 pasos de cada ejercicio

1. **Localiza los ids** en el HTML que te dan (`#btnMostrar` , `#mensaje` …). Si no coinciden con la plantilla, cámbialos.
2. **Pega la plantilla** del apartado correspondiente.
3. **Prueba el caso feliz y un caso malo** antes de pasar al siguiente.

REGLA DE ORO

Si tu código no hace nada, **el 90% de las veces es** `null` : el script se ejecutó antes de existir el HTML, o el `id` está mal escrito. Mira el apartado DOM antes de tocar nada más.

Qué evalúa cada ejercicio (para no regalar puntos)

Ejercicio	Lo que puntúa de verdad	Lo que NO puntúa
1 · Arrays	Que las funciones **devuelvan** (`return`) lo pedido: array, array, número.	Que imprimas por consola desde dentro de la función.
2 · Clases	Nombre de la clase, constructor, `mostrar()` con el **texto exacto** del enunciado.	Herencia, getters, validaciones extra.
3 · DOM	`createElement` + `appendChild` + `textContent` , y los dos eventos del ratón.	Bootstrap, animaciones, CSS bonito.
4 · Formulario	Regex válida + `preventDefault` + mensaje en el `div` .	Validar solo con HTML (`required` , `pattern`).

Ejercicio	Lo que puntúa de verdad	Lo que NO puntúa
5 · Storage	Guardar, ocultar el formulario, mostrar «Bienvenido de nuevo, X», botón y borrado.	Diseño, que recuerde la sesión de verdad.

01 Chuleta de 60 segundos

Si solo tienes cinco minutos antes del examen, lee esta tabla. Todo lo demás son variaciones de esto.

Quiero…	Código
Recorrer un array	`for (let i = 0; i < arr.length; i++) { arr[i] }` · o `arr.forEach(x => { ... })`
Filtrar	`arr.filter(x => x.categoria === "Drama")`
Buscar uno	`arr.find(x => x.id === 3)`
Comprobar si existe	`arr.some(x => x.titulo === "Dune")`
Sumar valores	`arr.reduce((suma, x) => suma + x.valoracion, 0)`
Media	`suma / arr.length`
Mayúsculas	`texto.toUpperCase()`
Comparar sin mayúsculas	`a.toLowerCase() === b.toLowerCase()`
Contiene texto	`texto.includes("lion")`
Quitar espacios	`texto.trim()`
Crear un elemento	`document.createElement("li")`
Meterlo en la página	`padre.appendChild(hijo)`
Poner texto	`el.textContent = "Matrix"`
Cambiar una clase	`el.classList.add("activo")` · `.remove()` · `.toggle()`
Cambiar un estilo	`el.style.fontWeight = "bold"`
Seleccionar un elemento	`document.getElementById("x")` · `document.querySelector("#x")`
Escuchar un clic	`btn.addEventListener("click", fn)`
Escuchar un formulario	`form.addEventListener("submit", e => { e.preventDefault(); ... })`

Quiero...	Código
Leer un input	`input.value.trim()`
Validar con regex	`/^[a-z]{3,50}$/i.test(valor)`
Guardar en el navegador	`localStorage.setItem("usuario", "Ana")`
Leer	`localStorage.getItem("usuario")` → `null` si no existe
Borrar una clave	`localStorage.removeItem("usuario")`
Guardar un objeto	`localStorage.setItem("carrito", JSON.stringify(arr))` · `JSON.parse(localStorage.getItem("carrito"))`
Ver errores en el navegador	`F12` → pestaña **Console**

---

02 Fundamentos que siempre caen

No hace falta dominar JavaScript entero. Esto es el 100% de lo que se usa en los cinco ejercicios.

Variables

```
let contador = 0;        // cambia de valor
const IVA = 0.21;        // no cambia (constante)
let titulo = "Dune";     // string
let anio = 2025;         // number
let visto = false;       // boolean

// Nunca uses una variable sin let/const: se crea una global y en modo
// estricto el navegador lanza ReferenceError.
```

> **ERROR CLÁSICO**
>
> `resultado = [];` dentro de una función **no es una variable local**: crea una global. Siempre `let resultado = [];`.

Comparar: `===`, no `==`

```
1 == "1"    // true  (convierte tipos, peligroso)
1 === "1"   // false (compara tipo y valor)
null == undefined   // true
null === undefined  // false
```

Valores que se comportan como «falso»

Valor	`if (valor)`	Comentario
`""`	falso	Input vacío → `!valor` es `true`
`"0"`	verdadero	¡Ojo! Un string con 0 no es falso
`0`	falso	
`null` / `undefined`	falso	Lo que devuelve `getElementById` si no existe
`NaN`	falso	Resultado de `Number("abc")`
`[]` / `{}`	verdadero	Un array vacío no es falso

Condicionales y bucles

```
if (nota >= 5) {
  console.log("Aprobado");
} else if (nota >= 3) {
  console.log("Suspenso justo");
} else {
  console.log("Suspenso");
}

// ternario: una sola instrucción
let estado = nota >= 5 ? "Aprobado" : "Suspenso";

// for clásico: el que siempre vale para arrays
for (let i = 0; i < peliculas.length; i++) {
  console.log(peliculas[i].titulo);
}

// for...of: más corto, sin índice
for (const p of peliculas) {
  console.log(p.titulo);
}

// forEach: con función
peliculas.forEach(function (p) {
  console.log(p.titulo);
});
```

Funciones

```
// Declaración (se puede llamar antes de definirla)
function sumar(a, b) {
  return a + b;
}

// Expresión (guarda la función en una variable)
const sumar = function (a, b) {
```

```
  return a + b;
};

// Arrow: lo mismo, más corto. Si es una sola línea, el return es implícito
const sumar = (a, b) => a + b;
const doble = n => n * 2;
const saludar = () => "Hola";

console.log(sumar(2, 3));   // 5
```

> **RETURN VS CONSOLE.LOG**
>
> Si el enunciado dice «devuelva», usa `return`. `console.log` solo *muestra*: no devuelve nada (devuelve `undefined`). Varios apartados del examen restan puntos si imprimes en vez de devolver.

Objetos y arrays

```
// Objeto literal
const pelicula = {
  titulo: "Dune: Parte Dos",
  director: "Denis Villeneuve",
  anio: 2025,
  categoria: "Ciencia Ficción",
  valoracion: 8.5
};

console.log(pelicula.titulo);        // notación punto
console.log(pelicula["director"]);   // notación corchete
pelicula.valoracion = 9;             // modificar
pelicula.vista = true;               // añadir propiedad

// Array
const numeros = [10, 20, 30];
console.log(numeros[0]);        // 10
console.log(numeros.length);    // 3
numeros.push(40);               // añadir al final
numeros.pop();                  // quitar el último

// Array de objetos (lo normal en el examen)
const peliculas = [
  { titulo: "Dune: Parte Dos", categoria: "Ciencia Ficción", valoracion: 8.5 },
  { titulo: "Inside Out 2", categoria: "Animación", valoracion: 7.8 }
];
```

Métodos de array que valen oro

Método	Qué hace	Ejemplo
`push(x)`	Añade al final	`arr.push(5)`
`pop()`	Quita el último	`arr.pop()`

Método	Qué hace	Ejemplo
`map(fn)`	Devuelve un array **nuevo** transformando cada elemento	`arr.map(x => x.titulo)`
`filter(fn)`	Devuelve un array **nuevo** con los que cumplen	`arr.filter(x => x.valoracion > 8)`
`find(fn)`	Devuelve el **primero** que cumple (o `undefined`)	`arr.find(x => x.titulo === "Dune")`
`findIndex(fn)`	Posición del primero que cumple (o `-1`)	`arr.findIndex(x => x.id === 2)`
`some(fn)`	`true` si **alguno** cumple	`arr.some(x => x.anio > 2020)`
`every(fn)`	`true` si **todos** cumplen	`arr.every(x => x.valoracion > 5)`
`reduce(fn, 0)`	Acumula un único valor	`arr.reduce((s, x) => s + x.valoracion, 0)`
`sort(fn)`	Ordena **mutando** el original	`[...arr].sort((a, b) => b.valoracion - a.valoracion)`
`slice(a, b)`	Copia una porción	`arr.slice(0, 3)`
`join(sep)`	Convierte a string	`["a","b"].join(" · ")` → `"a · b"`
`includes(x)`	¿Está ese **valor**?	`["a","b"].includes("a")`
`forEach(fn)`	Recorre sin devolver nada	`arr.forEach(x => console.log(x))`
`length`	Número de elementos	`arr.length`

> **INCLUDES NO SIRVE PARA BUSCAR DENTRO DE OBJETOS**
>
> `peliculas.includes("Dune")` es `false` aunque exista: el array contiene objetos, no strings. Para eso se usa `some` o `find` con una comparación de propiedad.

Plantillas de texto (template literals)

```
const nombre = "Ana";
const edad = 20;

console.log("Hola " + nombre);              // concatenación
console.log(`Hola ${nombre}, tienes ${edad}`);  // plantilla (más cómoda)

// ¡Cuidado! Son comillas invertidas: ` no es ' ni "
```

03 Ejercicio tipo 1 — Arrays de objetos

Te dan un array de objetos con propiedades fijas (`titulo` , `director` , `anio` , `categoria` , `valoracion`) y te piden 4 funciones. Todas menos la primera deben **devolver**.

El array de partida

```
let peliculas = [
  { titulo: "Dune: Parte Dos", director: "Denis Villeneuve", anio: 2025,
    categoria: "Ciencia Ficción", valoracion: 8.5 },
  { titulo: "Inside Out 2", director: "Kelsey Mann", anio: 2023,
    categoria: "Animación", valoracion: 7.8 },
  { titulo: "Mufasa: The Lion King", director: "Barry Jenkins", anio: 2024,
    categoria: "Aventura", valoracion: 8.0 }
];
```

> **PLANTILLA COMPLETA — LA DEL EXAMEN**
>
> Copia esto tal cual y cambia solo los nombres si el enunciado los cambia. Cumple los 4 apartados y no imprime desde dentro de las funciones.

```
// 1. Muestra por consola todos los títulos en mayúsculas (no devuelve nada)
function mayusculas() {
  for (let i = 0; i < peliculas.length; i++) {
    console.log(peliculas[i].titulo.toUpperCase());
  }
}

// 2. Nuevo array solo con las películas de esa categoría
function peliculasPorCategoria(categoria) {
  let resultado = [];
  for (let i = 0; i < peliculas.length; i++) {
    if (peliculas[i].categoria === categoria) {
      resultado.push(peliculas[i]);
    }
  }
  return resultado;   // [] si no hay ninguna
}

// 3. Películas cuyo título contiene el texto (sin distinguir mayúsculas)
function buscarPorTitulo(texto) {
  let resultado = [];
  for (let i = 0; i < peliculas.length; i++) {
    if (peliculas[i].titulo.toLowerCase().includes(texto.toLowerCase())) {
      resultado.push(peliculas[i]);   // la película, NO el título
    }
  }
  return resultado;
}

// 4. Media de todas las valoraciones
function mediaValoraciones() {
  let suma = 0;
  for (let i = 0; i < peliculas.length; i++) {
    suma += peliculas[i].valoracion;
```

```
  }
  return suma / peliculas.length;
}
```

La misma plantilla, versión corta (map / filter / reduce)

Si dominas estos tres métodos escribes la mitad. Valen exactamente igual:

```
const mayusculas = () => peliculas.forEach(p => console.log(p.titulo.toUpperCase()));

const peliculasPorCategoria = (categoria) =>
  peliculas.filter(p => p.categoria === categoria);

const buscarPorTitulo = (texto) =>
  peliculas.filter(p => p.titulo.toLowerCase().includes(texto.toLowerCase()));

const mediaValoraciones = () =>
  peliculas.reduce((suma, p) => suma + p.valoracion, 0) / peliculas.length;
```

> **ELIGE UNA Y NO LAS MEZCLES**
>
> La versión con `for` es más fácil de explicar si te preguntan; la de `filter` es más corta. Ambas puntúan. Lo que no puntúa es dejar una función a medias o devolver un `string` en vez de un array.

Variantes que pueden pedirte

Enunciado	Solución
Ordenar de mayor a menor valoración	`[...peliculas].sort((a, b) => b.valoracion - a.valoracion)`
Las 3 mejores	`[...peliculas].sort((a, b) => b.valoracion - a.valoracion).slice(0, 3)`
La película mejor valorada	`peliculas.reduce((mejor, p) => p.valoracion > mejor.valoracion ? p : mejor)`
Cuántas hay de cada categoría	`peliculas.reduce((cuenta, p) => { cuenta[p.categoria] = (cuenta[p.categoria] || 0) + 1; return cuenta; }, {})`
Buscar en título *o* director	`peliculas.filter(p => (p.titulo + " " + p.director).toLowerCase().includes(texto.toLowerCase()))`
Títulos en mayúsculas (array, no consola)	`peliculas.map(p => p.titulo.toUpperCase())`
Suma de valoraciones	`peliculas.reduce((s, p) => s + p.valoracion, 0)`

Enunciado	Solución
Películas entre dos años	`peliculas.filter(p => p.anio >= 2020 && p.anio <= 2025)`
Eliminar una película por título	`peliculas = peliculas.filter(p => p.titulo !== "Inside Out 2")`
Un array de strings «Título (año)»	`peliculas.map(p => \`${p.titulo} (${p.anio})\`)`

Errores que restan puntos aquí

Error	Por qué está mal	Correcto
Devolver `"No hay películas"` cuando el array sale vacío	El enunciado pide un array; un string no lo es	`return resultado;` → devuelve `[]`
`resultado.push(p.titulo)` en `buscarPorTitulo`	Devuelve strings, no películas	`resultado.push(p)`
`console.log(...)` dentro de una función que debe devolver	Pierdes 0,2 puntos por función	`return ...` y el `console.log` fuera
`resultado = []` sin `let`	Creas una variable global	`let resultado = []`
`peliculas.sort(...)` para ordenar	`sort` muta el array original	`[...peliculas].sort(...)`
`p.titulo = "Dune"` para comparar	Es una asignación, no una comparación	`p.titulo === "Dune"`
`p.titulo.includes("Lion")`	Distingue mayúsculas; el enunciado pide que no	`.toLowerCase().includes(texto.toLowerCase())`

CÓMO PROBARLO TÚ MISMO

Pon al final del archivo (fuera de las funciones) estas pruebas y ábrelo en la consola del navegador o ejecútalo con Node:

```
mayusculas();
console.log(peliculasPorCategoria("Animación"));   // 1 película
console.log(peliculasPorCategoria("Terror"));      // []
console.log(buscarPorTitulo("lion"));              // Mufasa...
console.log(mediaValoraciones());                  // 8.1
```

---

04 Ejercicio tipo 2 — Objetos y clases

Te piden una clase con un constructor, un método que devuelve una cadena con un formato **literal**, dos objetos, un array con ellos y recorrerlo mostrando el método.

PLANTILLA COMPLETA

```
class Pelicula {
  constructor(titulo, director, anio, categoria, valoracion) {
    this.titulo = titulo;
    this.director = director;
    this.anio = anio;
    this.categoria = categoria;
    this.valoracion = valoracion;
  }

  mostrar() {
    return `${this.titulo} (${this.anio}), dirigida por ${this.director}, en la categoría de
${this.categoria}`;
  }
}

// Dos objetos
let pelicula1 = new Pelicula("Dune: Parte Dos", "Denis Villeneuve", 2025, "Ciencia Ficción", 8.5);
let pelicula2 = new Pelicula("Mufasa: The Lion King", "Barry Jenkins", 2024, "Aventura", 8.0);

// Array y recorrido por consola
let peliculas = [pelicula1, pelicula2];

for (let i = 0; i < peliculas.length; i++) {
  console.log(peliculas[i].mostrar());
}
```

Salida:

```
Dune: Parte Dos (2025), dirigida por Denis Villeneuve, en la categoría de Ciencia Ficción
Mufasa: The Lion King (2024), dirigida por Barry Jenkins, en la categoría de Aventura
```

Anatomía de una clase

```
class NombreClase {
  constructor(parametro1, parametro2) {
    this.propiedad1 = parametro1;   // this = el objeto que se está creando
    this.propiedad2 = parametro2;
  }

  metodo() {
    return "algo usando " + this.propiedad1;
  }
}

const objeto = new NombreClase("a", "b");   // ¡new es obligatorio!
console.log(objeto.metodo());
```

Pieza	Qué hace
`class Pelicula { }`	Define el «molde». Por convención, mayúscula inicial.
`constructor(...)`	Se ejecuta al crear el objeto con `new`. Inicializa las propiedades.
`this.titulo`	La propiedad del objeto concreto. Sin `this` es una variable suelta.
`mostrar() { }`	Método: una función que vive dentro del objeto.
`new Pelicula(...)`	Crea un objeto nuevo. Sin `new` no hay objeto.

Variantes que pueden pedirte

Enunciado	Código
Método que devuelve el título en mayúsculas	`tituloMayus() { return this.titulo.toUpperCase(); }`
Método que devuelve si es recomendable	`esBuena() { return this.valoracion >= 7; }`
Método que devuelve un objeto	`resumen() { return { titulo: this.titulo, anio: this.anio }; }`
Contar cuántas hay	`console.log(peliculas.length)`
Recorrer con `forEach`	`peliculas.forEach(p => console.log(p.mostrar()))`
Filtrar los objetos de la clase	`peliculas.filter(p => p.valoracion > 8)`
Getter (propiedad calculada)	`get resumen() { return `${this.titulo} (${this.anio})`; }` → `p.resumen` sin paréntesis

Enunciado	Código
Método estático (no usa `this`)	`static saludar() { return "Hola"; }` → `Pelicula.saludar()`
Herencia simple	`class Serie extends Pelicula { constructor(t, d, a, c, v, temp) { super(t, d, a, c, v); this.temporadas = temp; } }`

EL FORMATO ES LITERAL

Si el enunciado dice `"en la categoría de X"`, no escribas `"en la categoría: X"`. Es un criterio de 0,5 puntos y se compara el texto. Copia la frase del enunciado y sustituye solo los valores.

Errores típicos

```
// ❌ Sin new: this apunta a undefined
let p = Pelicula("Dune", "Villeneuve", 2025, "Ciencia Ficción", 8.5);

// ❌ Comas en vez de punto y coma dentro de la clase
class A { constructor() { this.x = 1, this.y = 2 } }   // funciona, pero confunde

// ❌ Propiedad sin this: no se guarda en el objeto
constructor(titulo) { titulo = titulo; }

// ❌ Llamar al método sin paréntesis
console.log(p.mostrar);      // imprime la función, no el texto

// ✅
console.log(p.mostrar());
```

05 Ejercicio tipo 3 — DOM y eventos

ANTES DE ESCRIBIR UNA LÍNEA

Si tu JS «no hace nada» o da `Cannot read properties of null`, es esto: el `<script>` está en el `<head>` y se ejecuta **antes** de que existan los elementos. Solución en una palabra: `defer`.

```
<!-- ✅ así -->
<script src="dom.js" defer></script>

<!-- ✅ o al final del body -->
<body>
  ...
  <script src="dom.js"></script>
</body>

<!-- ❌ así: getElementById devuelve null -->
<head><script src="dom.js"></script></head>
```

Seleccionar elementos

Quiero	Código	Devuelve
Por id	`document.getElementById("listaPeliculas")`	1 elemento o `null`
Por selector CSS (id)	`document.querySelector("#listaPeliculas")`	el primero o `null`
Por selector CSS (clase)	`document.querySelector(".titulo")`	el primero o `null`
Todos por clase	`document.querySelectorAll(".item")`	lista (NodeList) — se recorre con `forEach`
Todos por etiqueta	`document.querySelectorAll("li")`	lista
Dentro de un elemento	`ul.querySelector("li")`	el primer `li` **de dentro**

> **EL ERROR QUE MÁS GENTE COMETE**
>
> `querySelector` devuelve **siempre el primero**. Si tienes 3 películas y haces `lista.querySelector("li")`, siempre te devuelve Matrix. Para el elemento concreto que ha recibido un evento se usa `e.target`.

Crear, meter y borrar elementos

```
// 1. Crear
let li = document.createElement("li");

// 2. Rellenar (texto plano, nunca HTML)
li.textContent = "Matrix";

// 3. Meter en la página
let ul = document.getElementById("listaPeliculas");
ul.appendChild(li);              // al final
ul.prepend(li);                  // al principio

// Clases y estilos
li.classList.add("activo");
li.classList.remove("activo");
li.classList.toggle("activo");   // si está la quita, si no la pone
li.classList.contains("activo"); // true / false
li.style.fontWeight = "bold";
li.style.color = "#b00";

// Atributos
li.setAttribute("data-id", "3");
li.id = "fila-3";

// Borrar
li.remove();
```

```
// Vaciar un contenedor (sin innerHTML)
while (ul.firstChild) {
  ul.removeChild(ul.firstChild);
}
```

> **TEXTCONTENT VS INNERHTML**
>
> `textContent` pone texto literal y es seguro. `innerHTML` interpreta HTML, y muchos enunciados **prohíben usarlo para crear elementos** («sin usar innerHTML o similares»). Usa siempre `createElement` + `textContent` + `appendChild`.

Eventos

Evento	Cuándo salta	Uso típico en el examen
`click`	Al pulsar un botón o elemento	Mostrar la lista
`submit`	Al enviar un formulario	Validar y `preventDefault()`
`input`	Cada vez que se escribe en un campo	Filtrar en vivo
`change`	Al cambiar y salir del campo / `select`	Detectar categoría elegida
`mouseover` / `mouseout`	Entrar / salir (burbujean)	Negrita al pasar el ratón
`mouseenter` / `mouseleave`	Entrar / salir (no burbujean)	Lo mismo, sobre un único elemento
`keydown`	Pulsar una tecla	Detectar `Enter` o `Escape`
`focus` / `blur`	Entrar / salir del campo	Validar al salir

```
// Estructura de un listener
boton.addEventListener("click", function () {
  // aquí va el código
});

// Con el objeto evento (necesario para saber qué elemento se pulsó)
lista.addEventListener("click", function (e) {
  console.log(e.target);      // el elemento concreto
});

// Con arrow function
boton.addEventListener("click", () => {
  console.log("pulsado");
});
```

Dos formas de asociar el efecto hover

A) Un listener por cada `li` (dentro del bucle):

```
for (let i = 0; i < peliculas.length; i++) {
  let li = document.createElement("li");
  li.textContent = peliculas[i].titulo;

  li.addEventListener("mouseover", function () {
    li.style.fontWeight = "bold";
  });
  li.addEventListener("mouseout", function () {
    li.style.fontWeight = "normal";
  });

  ul.appendChild(li);
}
```

B) Un único listener en la `<ul>`, aprovechando que el evento *burbujea* hasta el padre (delegación):

```
ul.addEventListener("mouseover", function (e) {
  if (e.target.tagName === "LI") {
    e.target.style.fontWeight = "bold";
  }
});

ul.addEventListener("mouseout", function (e) {
  if (e.target.tagName === "LI") {
    e.target.style.fontWeight = "normal";
  }
});
```

Plantilla completa del examen

```
let peliculas = [
  { titulo: "Matrix" },
  { titulo: "El Padrino" },
  { titulo: "Inception" }
];

const boton = document.getElementById("btnMostrar");
const ul = document.getElementById("listaPeliculas");

// 1 + 3: recorrer y crear los <li> sin innerHTML
boton.addEventListener("click", function () {
  while (ul.firstChild) ul.removeChild(ul.firstChild);  // evita duplicados

  for (let i = 0; i < peliculas.length; i++) {
    let li = document.createElement("li");
    li.textContent = peliculas[i].titulo;

    // 4: negrita al entrar
    li.addEventListener("mouseover", function () {
      li.style.fontWeight = "bold";
    });
    // 5: normal al salir
```

```
    li.addEventListener("mouseout", function () {
      li.style.fontWeight = "normal";
    });

    ul.appendChild(li);
  }
});
```

Variantes que pueden pedirte

Añadir un elemento desde un input

```
const input = document.getElementById("nuevoTitulo");

document.getElementById("agregar").addEventListener("click", function () {
  let texto = input.value.trim();
  if (texto === "") return;               // no añadir vacíos

  let li = document.createElement("li");
  li.textContent = texto;
  ul.appendChild(li);
  input.value = "";                       // limpiar el campo
});
```

Borrar un elemento (delegación + remove)

```
ul.addEventListener("click", function (e) {
  if (e.target.tagName === "LI") {
    e.target.remove();
  }
});
```

Marcar el seleccionado (quitar a los demás)

```
ul.addEventListener("click", function (e) {
  if (e.target.tagName !== "LI") return;

  let lis = ul.querySelectorAll("li");
  for (let i = 0; i < lis.length; i++) {
    lis[i].classList.remove("activo");
  }
  e.target.classList.add("activo");
});
```

Contador de clics

```
let clics = 0;

boton.addEventListener("click", function () {
  clics++;
```

```
  document.getElementById("contador").textContent = "Clics: " + clics;
});
```

Filtrar en vivo mientras se escribe

```
input.addEventListener("input", function () {
  let texto = input.value.toLowerCase();

  while (ul.firstChild) ul.removeChild(ul.firstChild);

  for (let i = 0; i < peliculas.length; i++) {
    if (peliculas[i].titulo.toLowerCase().includes(texto)) {
      let li = document.createElement("li");
      li.textContent = peliculas[i].titulo;
      ul.appendChild(li);
    }
  }
});
```

Mostrar / ocultar

```
caja.hidden = true;                    // oculto
caja.hidden = false;                   // visible
// equivalente con estilo:
caja.style.display = "none";
caja.style.display = "block";
```

Renderizar un array de objetos como lista

```
function pintarLista() {
  while (ul.firstChild) ul.removeChild(ul.firstChild);

  peliculas.forEach(function (p) {
    let li = document.createElement("li");
    li.textContent = p.titulo + " (" + p.anio + ")";
    ul.appendChild(li);
  });
}
```

Tabla a partir de un array

```
const tbody = document.querySelector("#tabla tbody");
while (tbody.firstChild) tbody.removeChild(tbody.firstChild);

peliculas.forEach(function (p) {
  let tr = document.createElement("tr");

  let tdTitulo = document.createElement("td");
  tdTitulo.textContent = p.titulo;

  let tdAnio = document.createElement("td");
  tdAnio.textContent = p.anio;
```

```
  tr.appendChild(tdTitulo);
  tr.appendChild(tdAnio);
  tbody.appendChild(tr);
});
```

Errores típicos de DOM

Error	Causa	Solución
`Cannot read properties of null`	Script antes del HTML o `id` mal escrito	`defer` + revisar el `id`
El hover solo funciona en el primero	`this.querySelector("li")` devuelve siempre el primero	`e.target`
Al pulsar dos veces se duplican los `<li>`	No vacías la lista antes de pintar	`while (ul.firstChild) ul.removeChild(ul.firstChild);`
`li is not defined`	Usar `var` en el bucle o sacar `li` del bucle	`let li` dentro del bucle
`appendChild is not a function`	El contenedor es `null` o un string	Selecciona el elemento padre correcto
El texto sale como `[object Object]`	Asignas el objeto entero a `textContent`	`li.textContent = p.titulo`
El botón recarga la página	Está dentro de un `<form>` y no haces `preventDefault`	`type="button"` o `e.preventDefault()`

---

06 Ejercicio tipo 4 — Validación de formularios

Siempre es el mismo esqueleto: escuchar el `submit`, leer los valores, comprobarlos con una regex y escribir el resultado en un `div`. La clave es **no enviar nunca el formulario**.

El esqueleto

```
const form = document.getElementById("formPelicula");
const mensaje = document.getElementById("mensaje");

form.addEventListener("submit", function (e) {
  e.preventDefault();        // 1. nunca se envía

  // 2. leer valores
```

```
  let titulo = document.getElementById("titulo").value.trim();

  // 3. validar
  let errores = [];
  if (!/^[a-záéíóúñü\s]{3,50}$/i.test(titulo)) {
    errores.push("Título: 3-50 letras.");
  }

  // 4. mostrar
  if (errores.length > 0) {
    mensaje.textContent = errores.join(" · ");
    mensaje.style.color = "red";
  } else {
    mensaje.textContent = "Película validada correctamente";
    mensaje.style.color = "green";
  }
});
```

Leer cada tipo de campo

Campo	Código	Nota
Texto	`input.value.trim()`	`trim()` quita espacios de los extremos
Número	`Number(input.value)`	Si está vacío devuelve `0`, cuidado
`<select>`	`select.value`	Vacío es `""`, no hace falta `trim`
Radio	`document.querySelector('input[name="genero"]:checked')`	Puede ser `null` si no hay ninguno marcado
Checkbox	`checkbox.checked`	`true` / `false`
Todos los de un formulario	`new FormData(form)`	Para casos avanzados

Regex: lo mínimo para aprobar

Símbolo	Significa	Ejemplo
`^`	Empieza por	`^Hola`
`$`	Termina en	`fin$`

Símbolo	Significa	Ejemplo
`.`	Cualquier carácter	`a.c` → abc, a1c
`[abc]`	Uno de esos caracteres	`[aeiou]`
`[a-z]`	Un rango	`[A-Za-z]` letras
`[^abc]`	Ninguno de esos	`[^0-9]` no dígito
`\d`	Un dígito (0-9)	`\d{4}` cuatro cifras
`\w`	Letra, número o `_`	`\w+`
`\s`	Espacio, tabulador o salto	`\s`
`*`	0 o más veces	`a*`
`+`	1 o más veces	`\d+`
`?`	0 o 1 vez (opcional)	`colou?r`
`{n}`	Exactamente n veces	`\d{4}`
`{n,m}`	Entre n y m veces	`{3,50}`
`(a|b)`	Una cosa u otra	`(19|20)\d{2}`
`\/i`	Flag: ignora mayúsculas	`/^[a-z]+$/i`
`\/g`	Flag: todas las coincidencias	`/\d/g`

Las regex del examen (copia estas)

Qué valida	Regex	Pasa	No pasa
Texto de 3 a 50 letras (con acentos y espacios)	`/^[a-záéíóúñü\s]{3,50}$/i`	"Matrix", "El niño"	"ab", "Matrix 2", 51 letras
Año de 4 cifras entre 1900 y 2099	`/^(19|20)\d{2}$/`	"1999", "2025"	"1899", "9999", "202"
Entero positivo	`/^\d+$/`	"0", "42"	"-1", "4.2"
Email sencillo	`/^[\w.+-]+@[\w-]+\.[a-z]{2,}$/i`	"ana@mail.com"	"ana@", "ana.com"
Usuario (letras y números, 4-12)	`/^[a-z0-9]{4,12}$/i`	"ana99"	"an", "ana_99"

Qué valida	Regex	Pasa	No pasa
Contraseña (mín. 8, letra y número)	`/^(?=.*[a-z])(?=.*\d).{8,}$/i`	"abc12345"	"abcdefgh"
Teléfono de 9 dígitos	`/^\d{9}$/`	"612345678"	"61234567"
Código postal (5 dígitos)	`/^\d{5}$/`	"48001"	"4800"
Solo letras, sin espacios	`/^[a-záéíóúñ]+$/i`	"Pepe"	"Pe pe"

> **CÓMO SE USA**
>
> `regex.test(valor)` devuelve `true` / `false`. Es lo que necesitas. `valor.match(regex)` devuelve la coincidencia o `null`; solo lo usarás si tienes que *extraer* algo.

Plantilla completa (versión corta, la recomendada)

```
const form = document.getElementById("formPelicula");
const mensaje = document.getElementById("mensaje");

const reTexto = /^[a-záéíóúñü\s]{3,50}$/i;   // letras, acentos y espacios
const reAnio = /^(19|20)\d{2}$/;             // 1900 - 2099

form.addEventListener("submit", function (e) {
  e.preventDefault();   // nunca se envía

  let titulo = document.getElementById("titulo").value.trim();
  let director = document.getElementById("director").value.trim();
  let anio = document.getElementById("anio").value.trim();
  let categoria = document.getElementById("categoria").value;

  let errores = [];
  if (!reTexto.test(titulo)) errores.push("Título: 3-50 letras.");
  if (!reTexto.test(director)) errores.push("Director: 3-50 letras.");
  if (!reAnio.test(anio)) errores.push("Año: 1900-2099.");
  if (categoria === "") errores.push("Elige categoría.");

  if (errores.length > 0) {
    mensaje.textContent = errores.join(" · ");
    mensaje.style.color = "red";
  } else {
    mensaje.textContent = "Película validada correctamente";
    mensaje.style.color = "green";
  }
});
```

Si prefieres un error por línea, cambia solo estas dos líneas:

```
mensaje.style.whiteSpace = "pre-line";
mensaje.textContent = errores.join("\n");
```

Versión con el campo marcado en rojo (si el profesor lo pide)

```
let campos = {
  titulo: document.getElementById("titulo"),
  director: document.getElementById("director"),
  anio: document.getElementById("anio"),
  categoria: document.getElementById("categoria")
};

form.addEventListener("submit", function (e) {
  e.preventDefault();

  // Limpiar marcas anteriores
  for (let nombre in campos) {
    campos[nombre].style.borderColor = "";
  }

  let errores = [];

  if (!reTexto.test(campos.titulo.value.trim())) {
    errores.push("Título incorrecto");
    campos.titulo.style.borderColor = "red";
  }
  if (!reTexto.test(campos.director.value.trim())) {
    errores.push("Director incorrecto");
    campos.director.style.borderColor = "red";
  }
  if (!reAnio.test(campos.anio.value.trim())) {
    errores.push("Año incorrecto");
    campos.anio.style.borderColor = "red";
  }
  if (campos.categoria.value === "") {
    errores.push("Elige categoría");
    campos.categoria.style.borderColor = "red";
  }

  if (errores.length > 0) {
    mensaje.textContent = errores.join(" · ");
    mensaje.style.color = "red";
  } else {
    mensaje.textContent = "Película validada correctamente";
    mensaje.style.color = "green";
  }
});
```

Variantes que pueden pedirte

Email + contraseña

```
const reEmail = /^[\w.+-]+@[\w-]+\.[a-z]{2,}$/i;
const rePass = /^(?=.*[a-z])(?=.*\d).{8,}$/i;

if (!reEmail.test(email)) errores.push("Email no válido.");
if (!rePass.test(pass)) errores.push("La contraseña necesita 8 caracteres, letra y número.");
```

Confirmar contraseña

```
if (pass !== pass2) {
  errores.push("Las contraseñas no coinciden.");
}
```

Edad mínima (número)

```
let edad = Number(document.getElementById("edad").value);

if (isNaN(edad) || edad < 18) {
  errores.push("Debes ser mayor de edad.");
}
```

Aceptar condiciones (checkbox)

```
if (!document.getElementById("condiciones").checked) {
  errores.push("Debes aceptar las condiciones.");
}
```

Validar al salir del campo (no al enviar)

```
document.getElementById("titulo").addEventListener("blur", function () {
  this.style.borderColor = reTexto.test(this.value.trim()) ? "" : "red";
});
```

Formulario que sí se envía cuando todo está bien

```
form.addEventListener("submit", function (e) {
  if (!todoCorrecto()) {
    e.preventDefault();       // solo bloqueas si hay errores
    mensaje.textContent = "Corrige los errores";
  }
  // si no hay errores, el navegador envía el formulario
});
```

> **NO PIERDAS LOS 2 PUNTOS POR ESTO**
>
> Si añades `required`, `pattern` o `minlength` al HTML, el navegador puede bloquear el envío **antes** de que tu JS valide, y no verás tus mensajes. Añade `novalidate` al `<form>` y valida solo con JS: es lo que pide el enunciado («no se valorará validación realizada únicamente con HTML5»).

Errores típicos de validación

Error	Consecuencia	Arreglo
Sin `e.preventDefault()`	La página se recarga y el mensaje desaparece	Añadirlo como primera línea del `submit`
`if (!/^\d{4}$/.test(anio))` y ya está	Acepta `0000` y `9999`	Añadir el rango: `/^(19|20)\d{2}$/`
`/[a-z]{3,50}/` sin `^ $`	«ab12345» pasa porque encuentra una parte válida	Anclar: `/^[a-z]{3,50}$/i`
Usar `regex.test()` con flag `g` en bucle	Resultados alternos (la última posición se guarda)	Sin `g` para validar
Comparar `valor.length > 3` en vez de regex	El enunciado exige regex explícitamente	`{3,50}` en la regex
`mensaje.innerHTML = ...`	Funciona, pero algunos correctores lo marcan	`textContent`
No vaciar `mensaje`	Los errores se acumulan en cada intento	Asignar `textContent` (ya borra) o vaciar antes
Leer `.value` sin `.trim()`	`" "` pasa como válido	`.value.trim()`

---

07 Ejercicio tipo 5 — Web Storage

Se guarda un dato en el navegador y se usa para decidir qué se muestra. En el examen es un login: si hay usuario guardado, se oculta el formulario y se muestra «Bienvenido de nuevo, X» con un botón de cerrar sesión.

La API completa (son 4 métodos)

Método	Qué hace	Devuelve
`localStorage.setItem("clave", valor)`	Guarda o sobrescribe	`undefined`
`localStorage.getItem("clave")`	Lee	El string, o `null` si no existe
`localStorage.removeItem("clave")`	Borra esa clave	`undefined`
`localStorage.clear()`	Borra **todo**	`undefined`
`localStorage.length`	Cuántas claves hay	número

Método	Qué hace	Devuelve
`localStorage.key(0)`	Nombre de la clave en esa posición	string

> **TRES REGLAS QUE SE OLVIDAN**
>
> 1. Solo guarda **texto**. Un objeto se convierte en `"[object Object]"` si no usas `JSON.stringify`.
> 2. Los datos son **por origen** (dominio + puerto): lo que guardas en un sitio no existe en otro.
> 3. Persiste al recargar y al cerrar el navegador (eso es `localStorage`; `sessionStorage` muere al cerrar la pestaña).

Plantilla completa (la del examen)

```
const form = document.getElementById("formUsuario");
const bienvenida = document.getElementById("bienvenida");
const cerrar = document.getElementById("cerrarSesion");

// Pinta la página según haya usuario guardado o no
function pintar() {
  const usuario = localStorage.getItem("usuario");

  if (usuario) {
    form.style.display = "none";                              // oculta el formulario
    bienvenida.textContent = "Bienvenido de nuevo, " + usuario;
    cerrar.hidden = false;                                     // muestra el botón
  } else {
    form.style.display = "block";                              // vuelve el formulario
    bienvenida.textContent = "";
    cerrar.hidden = true;
  }
}

// Login: guarda el usuario y repinta
form.addEventListener("submit", function (e) {
  e.preventDefault();                          // no recargar la página
  const usuario = document.getElementById("nombreUsuario").value.trim();

  if (usuario !== "") {
    localStorage.setItem("usuario", usuario);
    pintar();
  }
});

// Cerrar sesión: borra y repinta
cerrar.addEventListener("click", function () {
  localStorage.removeItem("usuario");
  pintar();
});

// Estado inicial al cargar
pintar();
```

En el HTML hacen falta el `<script defer>` y el botón:

```
<h1 id="bienvenida"></h1>

<div id="contenedor">
  <form id="formUsuario">
    <label>Usuario: <input type="text" id="nombreUsuario"></label>
    <button type="submit">Login</button>
  </form>

  <button id="cerrarSesion" hidden>Cerrar sesión</button>
</div>
```

> **LA IDEA QUE HAY QUE ENTENDER**
>
> Todo se reduce a una función `pintar()` que **lee el storage y decide qué se ve**. La llamas al cargar, después de guardar y después de borrar. Así el login y el logout no duplican lógica y no se te olvida ningún caso.

Variantes que pueden pedirte

Guardar un objeto o un array (JSON)

```
const pelicula = { titulo: "Dune", anio: 2025 };

// Guardar
localStorage.setItem("pelicula", JSON.stringify(pelicula));

// Leer
const guardada = JSON.parse(localStorage.getItem("pelicula"));
console.log(guardada.titulo);   // "Dune"

// Array
localStorage.setItem("favoritas", JSON.stringify(["Dune", "Matrix"]));
const favoritas = JSON.parse(localStorage.getItem("favoritas")) || [];
favoritas.push("Inception");
localStorage.setItem("favoritas", JSON.stringify(favoritas));
```

Contador de visitas

```
let visitas = Number(localStorage.getItem("visitas")) || 0;
visitas++;
localStorage.setItem("visitas", visitas);

document.getElementById("contador").textContent = "Visitas: " + visitas;
```

Recordar el nombre aunque no haya login

```
const nombre = localStorage.getItem("nombre");
if (nombre) {
  document.getElementById("saludo").textContent = "Hola otra vez, " + nombre;
}
```

Guardar una preferencia (tema oscuro)

```
const boton = document.getElementById("tema");

boton.addEventListener("click", function () {
  document.body.classList.toggle("oscuro");
  localStorage.setItem("tema", document.body.classList.contains("oscuro") ? "oscuro" : "claro");
});

// Al cargar
if (localStorage.getItem("tema") === "oscuro") {
  document.body.classList.add("oscuro");
}
```

Carrito simple (array de objetos)

```
function leerCarrito() {
  return JSON.parse(localStorage.getItem("carrito")) || [];
}

function guardarCarrito(carrito) {
  localStorage.setItem("carrito", JSON.stringify(carrito));
}

function agregar(nombre) {
  let carrito = leerCarrito();
  carrito.push({ nombre: nombre });
  guardarCarrito(carrito);
  pintarCarrito();
}

function vaciar() {
  localStorage.removeItem("carrito");
  pintarCarrito();
}
```

sessionStorage (solo dura la pestaña)

```
sessionStorage.setItem("paso", "2");
sessionStorage.getItem("paso");
sessionStorage.removeItem("paso");
```

Comprobar si existe sin leer dos veces

```
const usuario = localStorage.getItem("usuario");
if (usuario !== null) {
  // existe (ojo: un string vacío también existe)
}

// o simplemente, si nunca guardas vacíos:
if (usuario) {
  // existe y no está vacío
}
```

Errores típicos

Error	Qué pasa	Arreglo
`localStorage.setItem("x", objeto)`	Guarda `"[object Object]"`	`JSON.stringify(objeto)`
`localStorage.getItem("x")` y usar el número directamente	`"3" + 1` da `"31"`	`Number(...)`
Guardar pero no repintar	El usuario no ve el cambio hasta recargar	Llamar a `pintar()` después de guardar/borrar
Ocultar el formulario pero no volver a mostrarlo	Al cerrar sesión no hay forma de entrar	`form.style.display = "block"` en el `else`
Comparar con `=== ""` cuando no existe	`getItem` devuelve `null`, no `""`	`if (usuario)` o `!== null`
Guardar sin `preventDefault`	La página se recarga con el usuario en la URL	`e.preventDefault()`
Probar con `file://` y cambiar de carpeta	Cada archivo puede ser un «origen» distinto	Probar con un servidor local o desde la misma ruta

---

08 Modo examen: plan, orden y tiempos

Los apartados son independientes: si uno se atasca, no bloquea los demás. Esta es la estrategia que maximiza la nota en el tiempo que hay.

Orden recomendado

Orden	Ejercicio	Tiempo	Por qué
1º	Clases (Ej. 2)	10 min	Es el más corto y mecánico: 1,5 puntos casi regalados.
2º	Arrays (Ej. 1)	20 min	Cuatro funciones con `for`; lo único delicado es el `return`.
3º	Web Storage (Ej. 5)	12 min	Una función `pintar()` y dos listeners.
4º	DOM (Ej. 3)	15 min	Si te acuerdas del `defer`, sale solo.

Orden	Ejercicio	Tiempo	Por qué
5º	Validación (Ej. 4)	15 min	Es donde más gente se lía con las regex; déjalo para el final con la cabeza fresca.

> **ANTES DE ENTREGAR**
>
> Ejecuta cada archivo y comprueba el caso feliz **y** un caso malo. Un ejercicio que «parece bien» pero lanza un error en consola no puntúa. Con 5 minutos de pruebas se salvan puntos enteros.

Cómo probar cada ejercicio en 30 segundos

Ejercicio	Prueba	Debe pasar
1 · Arrays	`console.log(buscarPorTitulo("lion"))`	Array con la película; si no hay, `[]`
2 · Clases	Ejecutar el archivo	Dos líneas con el formato exacto del enunciado
3 · DOM	Pulsar el botón 2 veces y pasar el ratón	3 `<li>` (no 6) y negrita en cualquiera
4 · Formulario	Enviar vacío y luego relleno	Errores en rojo / «Película validada correctamente», sin recargar
5 · Storage	Login, F5, cerrar sesión	Mantiene la sesión al recargar y vuelve el formulario

Si te atascas

1. **Abre la consola (F12)**. El mensaje en rojo dice la línea exacta. No adivines.
2. Si dice `null` → revisa `id` y `defer`.
3. Si no pasa nada al pulsar → comprueba que el listener está fuera de otra función y que el script cargó (un error anterior lo para todo).
4. Si la validación no salta → comprueba que escuchas `submit` en el `form`, no `click` en el botón.
5. Deja ese apartado, haz otro y vuelve. Los cinco son independientes.

Lo que NO debes hacer

- Usar `innerHTML` para crear elementos en el ejercicio de DOM (varios enunciados lo prohíben explícitamente).
- `console.log` dentro de funciones que deben devolver.
- Validar solo con `required` / `pattern` del HTML.
- Cambiar los `id` del HTML que te dan «para que quede más bonito».
- Entregar sin comprimir o sin subir a Moodle. Comprime la carpeta entera con los archivos originales.

09 Errores que más puntos cuestan

Esta tabla es tu diccionario cuando algo falla. Busca el mensaje en la columna izquierda.

Mensaje / síntoma	Causa	Solución
`Cannot read properties of null (reading 'addEventListener')`	El elemento no existe cuando corre el script: falta `defer` o el `id` está mal	`<script defer>` y revisar el `id`
`x is not defined`	Variable no declarada o con otro nombre (mayúsculas incluidas)	`let x = ...` y revisar la ortografía
`x is not a function`	Llamas a algo que no es función: `null`, un string, o un método mal escrito	Comprobar el nombre y que el elemento exista
`Unexpected token` / el archivo entero no se ejecuta	Error de sintaxis: falta una llave, paréntesis o comilla; `!===`; coma final	Leer la línea que marca la consola
`undefined` en un `console.log`	La función no tiene `return` (solo imprime)	Añadir `return`
`NaN`	Operas con texto: `"8.5" * 2` funciona, pero `Number("abc")` no	`Number()` y comprobar con `isNaN()`
`[object Object]`	Metes un objeto donde va texto	`p.titulo`, o `JSON.stringify(p)`
El array devuelto está vacío cuando debería tener algo	Comparas con `=` (asignación), con `==`, o con distinta capitalización/acentos	`===` y `toLowerCase()` en ambos lados
El formulario «no hace nada» y se recarga	Falta `e.preventDefault()` o escuchas `click` y no `submit`	`form.addEventListener("submit", e => { e.preventDefault(); ... })`
El mensaje de error se queda pegado	No limpias el `div`	`mensaje.textContent = ""` al empezar
Al pulsar el botón se duplican los elementos	No vacías la lista antes de pintar	`while (ul.firstChild) ul.removeChild(ul.firstChild);`

Mensaje / síntoma	Causa	Solución
El hover solo cambia el primer elemento	`querySelector("li")` siempre devuelve el primero	`e.target` en el listener del padre
Los acentos salen raros (`Ã³`)	Archivo guardado en otra codificación	Guardar como UTF-8 y `<meta charset="UTF-8">`
`localStorage` no guarda nada	Objeto sin `JSON.stringify` , o pruebas en otro origen	Ver el error típico del apartado 07

Comprobación rápida de sintaxis

Si tienes Node instalado, `node --check archivo.js` te dice si hay un error de sintaxis sin ejecutar nada. En el navegador, la consola lo dice directamente al cargar.

---

10 Checklist final imprimible

Marca cada casilla antes de comprimir y subir el examen.

Generales

- [ ] Todos los archivos guardados con los nombres originales.
- [ ] Consola del navegador sin errores en rojo en ninguna página.
- [ ] Los `<script>` llevan `defer` o están al final del `<body>` .
- [ ] Los `id` que uso existen en el HTML.
- [ ] He probado cada ejercicio con un caso correcto y uno incorrecto.
- [ ] Carpeta comprimida (ZIP) y subida a la tarea de Moodle.

1 · Arrays

- [ ] `mayusculas()` imprime los títulos en mayúsculas.
- [ ] `peliculasPorCategoria(c)` devuelve un **array** (vacío si no hay).
- [ ] `buscarPorTitulo(t)` devuelve **películas**, no títulos.
- [ ] La búsqueda ignora mayúsculas y minúsculas.
- [ ] `mediaValoraciones()` devuelve un número.
- [ ] Ninguna función imprime por consola desde dentro (salvo `mayusculas`).

2 · Clases

- [ ] Clase `Pelicula` con los 5 atributos.
- [ ] Constructor que inicializa con `this.`.
- [ ] `mostrar()` devuelve el texto con el formato **literal** del enunciado.
- [ ] Dos objetos creados con `new`.
- [ ] Los dos objetos están en un array.
- [ ] Se recorre el array e imprime `mostrar()`.

3 · DOM y eventos

- [ ] Listener de `click` asociado a `#btnMostrar`.
- [ ] Recorro el array de películas.
- [ ] Creo cada `<li>` con `createElement` y `textContent` (sin `innerHTML`).
- [ ] Los meto en `#listaPeliculas` con `appendChild`.
- [ ] Al pasar el ratón se pone en negrita.
- [ ] Al salir vuelve a la normalidad.
- [ ] Si pulso dos veces no se duplican.

4 · Validación

- [ ] Escucho `submit` en el formulario.
- [ ] Llamo a `e.preventDefault()` siempre.
- [ ] Título y director: regex de 3 a 50 letras.
- [ ] Año: 4 dígitos entre 1900 y 2099.
- [ ] Categoría no vacía.
- [ ] Muestro todos los errores juntos o uno por uno.
- [ ] Si todo es correcto: «Película validada correctamente» en `#mensaje`.
- [ ] Leo los valores con `.value.trim()`.

5 · Web Storage

- [ ] Al enviar el formulario guardo el usuario en `localStorage`.
- [ ] Si existe, oculto el formulario.
- [ ] Si existe, muestro «Bienvenido de nuevo, `<usuario>`».
- [ ] Si existe, muestro el botón «Cerrar sesión».
- [ ] Al cerrar sesión borro la clave del storage.
- [ ] Al cerrar sesión vuelve a aparecer el formulario.
- [ ] Al recargar la página se mantiene el estado.

ÚLTIMO CONSEJO

En el examen no improvises nombres ni estructuras: **reconoce el patrón**, pega la plantilla, adapta los `id` y los textos, y prueba. Los cinco ejercicios son variaciones de lo que hay en esta guía.